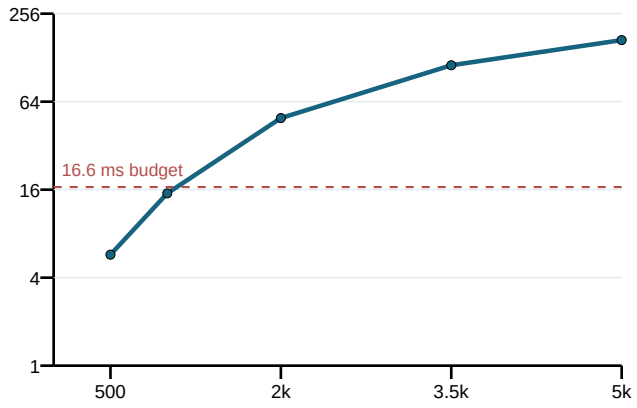
Tidekeeper: measured scale

Niagara at Scale / Track B / UE 5.7.4 / October 26, 2026

Declared budget: 1920x1080, 60 fps (16.6 ms), Niagara game + render thread at most 4 ms. Targets: 500 complex CPU mesh agents and 100,000 simple GPU sprites. This report uses p95 frame time for sustained-frame acceptance.

Development machine: i9-14900HX, RTX 4080 Laptop GPU, 64 GB RAM, Windows 11. Windows Development build, DX12/SM6, Epic scalability, unlit arena, Lumen/ray tracing and VSync/frame cap off. These are local results; lab validation is outstanding.

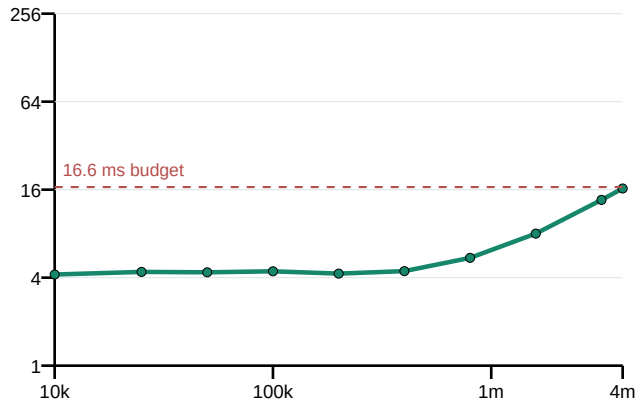
CPU / spatial hash



Particle count; mean frame time in ms (log y)

CPU count	Mean ms	p95 ms	GT+RT ms
500	5.75	7.38	3.09
1k	15.09	18.33	11.96
2k	49.42	58.88	46.60
3.5k	113.54	135.33	111.37
5k	168.79	192.75	167.09

GPU / bounded neighbor grid



Particle count; mean frame time in ms (log y)

GPU count	Mean ms	p95 ms	Compute ms
10k	4.20	4.70	0.04
25k	4.38	4.99	0.05
50k	4.35	5.08	0.07
100k	4.42	5.21	0.14
200k	4.26	5.02	0.25
400k	4.43	5.29	0.58
800k	5.46	6.34	1.20
1.6m	8.01	9.02	2.41
3.2m	13.62	15.24	4.89
4m	16.30	18.16	6.10

Targets: CPU 500: p95 7.38 ms, Niagara GT+RT 3.09 ms. GPU 100k: p95 5.21 ms, GT+RT 0.06 ms, GPU compute 0.14 ms.

First sampled budget failures: CPU 1k; GPU 4m. CPU and GPU limits are measured separately; they are not added as if the threads execute serially.

Method: five CPU counts before/after hashing; ten GPU counts. Each stage uses four seconds of warmup and five seconds of samples with a fixed seed, camera and timed attractor route. Actual Niagara particle counts were verified. stat Niagara/stat GPU overlays and named CPU/GPU Insights regions were captured.

Timing definitions: GT/worker/RT values come from engine component particle-performance counters. GT+RT additionally includes separately traced custom GPU-DI render-thread setup. GPU compute is the outer Niagara GPU dispatch scope, which encloses the custom DI kernels. It excludes sprite drawing. Raw CSVs preserve all counters.

Source: Docs/Results/Measurements.csv; raw measurements and trace exports supplied alongside it.

Optimization and game trade-offs

A measured improvement is conditional on distribution, not just particle count.

Optimization: replace all-pairs CPU neighbor visits with an exact spatial hash. Rules, perception radius, seed, arena and test route stay the same. An automation test checks matching trajectories at the same timesteps. The table shows custom CPU-solver time; negative gains mean a regression.

CPU count	All pairs (ms)	Hash (ms)	Reduction
500	3.63	3.04	+16.4%
1k	14.17	11.87	+16.2%
2k	53.06	46.41	+12.5%
3.5k	127.26	110.80	+12.9%
5k	166.11	166.04	+0.0%

Interpretation: hashing reduces solver time by 16.4% at 500 agents, but the gain approaches zero at 5,000. A dense following cluster puts most neighbors into a few cells; traversal overhead consumes the expected gain. The next optimization should reduce dense-cell work without hiding a behavior change.

Two-way gameplay coupling: input changes attraction/repulsion; simulation positions decide rescue. CPU returns the rescued total directly. GPU reduces rescued/overflow counts and asynchronously reads back only two uint32 values every sixth tick. Run IDs and monotonic serials reject late callbacks after a restart. Captured agents remain rescued.

GPU quality cost: the 25x15 grid stores at most 64 indices per cell; an agent samples up to eight entries in each adjacent cell. In the dense 100k test, 99.5% of agents overflow cell storage at the final sample. They still move and render, but are absent from stored neighbor lists. This severe approximation is displayed in the HUD/CSV and is not an exact all-neighbor CPU comparison.

At similar frame time: CPU 500 averages 5.75 ms; the closest measured GPU configuration, 800k, averages 5.46 ms (difference 0.28 ms). This is a measured near-match, not an assertion of identical frame times. The GPU supports a much denser school; the CPU gives individually readable state colors, leaders and obstacle avoidance.

Design choice: keep the CPU configuration as the default for this small herding arena because individual feedback and obstacle response explain the player's decisions. Keep GPU mode for scale and collective motion. This is a design argument from implemented behavior and rendering, not a user-preference study.

Limits: one laptop, one dense-follow route and one sampling window. At overloaded counts, the 33.3-ms simulation step cap slows simulated motion relative to wall time, so trajectories are not identical across different frame rates. Tracing/stat overlays add overhead. Lab performance and physical mouse/keyboard playtesting remain separate checks.

With another week: test a hierarchical/density-aware neighbor scheme, improve sprite readability and feedback, calibrate an exact equal-frame-time pair, collect player observations, and rerun the same captures on the lab hardware.